

PART 1

Import Clients

```
LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row
CREATE (:Client {
    id: row.id,
    name: row.name,
    email: row.email,
    phone: row.phone,
    tfn: row.tfn,
    flagged: row.flagged
});
```

Import Stores

```
LOAD CSV WITH HEADERS FROM 'file:///stores.csv' AS row
CREATE (:Store {
    id: row.id,
    name: row.name
});
```

Verify IDs existing

```
MATCH (c:Client) RETURN c.id, c.name LIMIT 5;
MATCH (s:Store) RETURN s.id, s.name LIMIT 5;
```

Relationship creation query

```
LOAD CSV WITH HEADERS FROM 'file:///purchase.csv' AS row
MATCH (c:Client {id: row.idFrom})
MATCH (s:Store {id: row.idTo})
CREATE (c)-[:PURCHASED {
    amount: toFloat(row.amount),
    timeOffset: toInteger(row.timeOffset)
}]->(s);
```

Check the relationships (Connections between Clients and Stores)

```
MATCH ()-[r:PURCHASED]->() RETURN r LIMIT 10;
```

(Client → Client) Transfer Relationships

```
LOAD CSV WITH HEADERS FROM 'file:///xfer.csv' AS row
MATCH (sender:Client {id: row.idFrom})
MATCH (receiver:Client {id: row.idTo})
CREATE (sender)-[:TRANSFERRED_TO {
    amount: toFloat(row.amount),
    timeOffset: toInteger(row.timeOffset)
}]->(receiver);
```

Confirm transfers created

```
MATCH ()-[r:TRANSFERRED_TO]->() RETURN r LIMIT 10;
```

Check total relationship summary

```
MATCH ()-[r]->() RETURN DISTINCT type(r), count(*) LIMIT 10;
```

	Table	RAW
	type(r)	count(*)
1	"PURCHASED"	35746
2	"TRANSFERRED_TO"	9626

Visualize the full network

```
MATCH (a)-[r]->(b)
```

```
RETURN a.name AS From, type(r) AS Relationship, b.name AS To, r.amount AS Amount
```

```
LIMIT 50;
```

PURCHASED and TRANSFERRED_TO

```
LOAD CSV WITH HEADERS FROM 'file:///purchase.csv' AS row
```

```
MATCH (c:Client {id: row.idFrom})
```

```
MATCH (s:Store {id: row.idTo})
```

```
CREATE (t:Transaction:Purchase {
```

```
  txn_id: apoc.create.uuid(),
```

```
  amount: toFloat(row.amount),
```

```
  timestamp: datetime({epochMillis: toInteger(row.timeOffset)}),
```

```
  hour: toInteger(apoc.date.format(toInteger(row.timeOffset), 's', 'H')),
```

```
  weekday: apoc.date.format(toInteger(row.timeOffset), 's', 'EEE'),
```

```
  business_hours:
```

```
    CASE
```

```
      WHEN toInteger(apoc.date.format(toInteger(row.timeOffset), 's', 'H')) >= 9
```

```
        AND toInteger(apoc.date.format(toInteger(row.timeOffset), 's', 'H')) <= 17
```

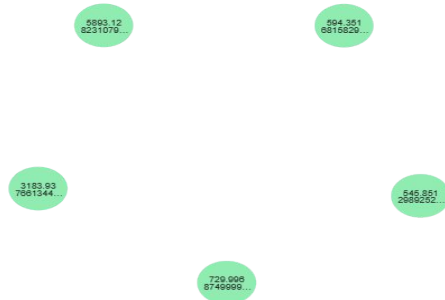
```
      THEN true
```

```
      ELSE false
```

```

END
}))
CREATE (c)-[:MADE]->(t)
CREATE (t)-[:AT_STORE]->(s);

```



Create Transfer Transactions

```

LOAD CSV WITH HEADERS FROM 'file:///xfer.csv' AS row
MATCH (sender:Client {id: row.idFrom})
MATCH (receiver:Client {id: row.idTo})
CREATE (t:Transaction:Transfer {
  txn_id: apoc.create.uuid(),
  amount: toFloat(row.amount),
  timestamp: datetime({epochMillis: toInteger(row.timeOffset)}),
  hour: toInteger(apoc.date.format(toInteger(row.timeOffset), 's', 'H')),

```

```
weekday: apoc.date.format(toInteger(row.timeOffset), 's', 'EEE'),
```

```
business_hours:
```

```
CASE
```

```
  WHEN toInteger(apoc.date.format(toInteger(row.timeOffset), 's', 'H')) >= 9
```

```
  AND toInteger(apoc.date.format(toInteger(row.timeOffset), 's', 'H')) <= 17
```

```
  THEN true
```

```
  ELSE false
```

```
END
```

```
}}
```

```
CREATE (sender)-[:SENT]->(t)
```

```
CREATE (t)-[:TO]->(receiver);
```

```
// Check one transfer transaction
```

```
MATCH (t:Transaction:Transfer) RETURN t LIMIT 5;
```

```
// See sample edges
```

```
MATCH (a:Client)-[r:SENT]->(t:Transfer)-[:TO]->(b:Client)
```

```
RETURN a.name AS Sender, b.name AS Receiver, t.amount AS Amount, t.business_hours  
AS BusinessHours
```

```
LIMIT 10;
```

NEXT_TXN_OF relationships

```
// represent transaction sequences per client
```

```
MATCH (c:Client)-[:MADE|:SENT]->(t:Transaction)
```

```
WITH c, t ORDER BY t.timestamp
```

```
WITH c, collect(t) AS txns
```

```
UNWIND range(0, size(txns)-2) AS i
```

```
WITH txns[i] AS t1, txns[i+1] AS t2, c
```

```
MERGE (t1)-[:NEXT_TXN_OF {client_id: c.id}]->(t2);
```

SIMILAR_TO between clients

```
MATCH (c1:Client)
WITH c1
MATCH (c2:Client)
WHERE c1.tfn = c2.tfn OR c1.email = c2.email
AND c1 <> c2
MERGE (c1)-[:SIMILAR_TO]->(c2);
```

Add :Source Nodes and Link to Transactions

```
// Create Source nodes
MERGE (s1:Source {name:'purchase.csv'})
MERGE (s2:Source {name:'xfer.csv'});

// Link Purchase transactions to their source
MATCH (t:Purchase), (s:Source {name:'purchase.csv'})
MERGE (t)-[:FROM_SOURCE]->(s);

// Link Transfer transactions to their source
MATCH (t:Transfer), (s:Source {name:'xfer.csv'})
MERGE (t)-[:FROM_SOURCE]->(s);
```

Add Constraints and Indexes (for Scalability and Integrity)

```
// Ensure unique IDs for Clients, Stores, and Transactions
CREATE CONSTRAINT unique_client_id IF NOT EXISTS
FOR (c:Client) REQUIRE c.id IS UNIQUE;

CREATE CONSTRAINT unique_store_id IF NOT EXISTS
FOR (s:Store) REQUIRE s.id IS UNIQUE;
```

```
CREATE CONSTRAINT unique_txn_id IF NOT EXISTS
```

```
FOR (t:Transaction) REQUIRE t.txn_id IS UNIQUE;
```

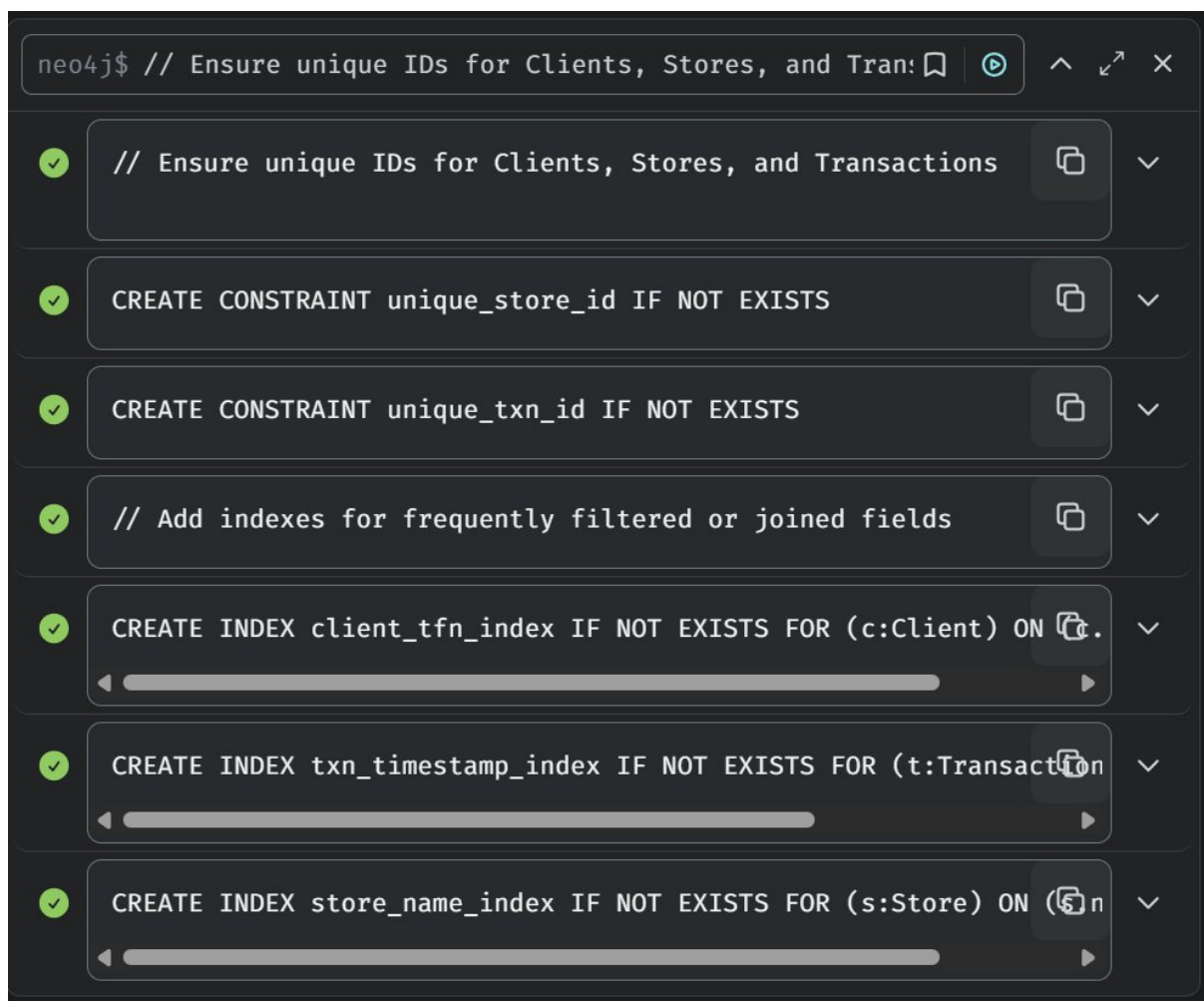
```
// Add indexes for frequently filtered or joined fields
```

```
CREATE INDEX client_email_index IF NOT EXISTS FOR (c:Client) ON (c.email);
```

```
CREATE INDEX client_tfn_index IF NOT EXISTS FOR (c:Client) ON (c.tfn);
```

```
CREATE INDEX txn_timestamp_index IF NOT EXISTS FOR (t:Transaction) ON  
(t.timestamp);
```

```
CREATE INDEX store_name_index IF NOT EXISTS FOR (s:Store) ON (s.name);
```



To compare query performance *with* and *without* indexes:

```
// Without Index
```

```
DROP INDEX txn_timestamp_index;
```

```
PROFILE MATCH (t:Transaction) WHERE t.timestamp >= datetime('2025-01-01')
RETURN count(t);
```

// With Index

```
CREATE INDEX txn_timestamp_index IF NOT EXISTS FOR (t:Transaction) ON
(t.timestamp);
```

```
PROFILE MATCH (t:Transaction) WHERE t.timestamp >= datetime('2025-01-01')
RETURN count(t);
```

Explanation

The payments graph was designed for scalability, query efficiency, and extensibility in future fraud-detection research.

1. Modeling Extensions

Two new relationship types were added:

- NEXT_TXN_OF links each client's consecutive transactions in time, supporting temporal sequence modeling (e.g., bursts of small transfers before a large fraud).
 - SIMILAR_TO connects clients sharing TFNs or emails, revealing potential collusion, duplicate accounts, or household clusters.
- Both relationships enable higher-level pattern detection through Neo4j Graph Data Science algorithms .

2. Temporal Representation

The original timeOffset was converted to a datetime field (timestamp) with derived features such as hour, weekday, and business_hours. This supports fraud behaviour rules.

3. Constraints and Indexes

Unique constraints on IDs for Client, Store, and Transaction guarantee entity integrity across imports. Indexes on email, tfn, and timestamp reduce lookup costs for similarity joins and temporal aggregation queries.

4. Data Integrity and Provenance

Each dataset (purchase.csv, xfer.csv) was represented by a Source node connected via FROM_SOURCE relationships to its transactions. This design preserves data provenance, allowing auditing and multi-source expansion without schema changes.

5. Extensibility

Both Purchase and Transfer are modeled as sub-types of Transaction, allowing future types like Refund, Loan, or CryptoTransfer to reuse the same structure.

PART 2

Problem 1:

Top Spenders Between a Time Window

```
// ----- PARAMETERS -----
```

```
:param startTime => datetime("2024-05-12T10:00:00+10:00");
```

```
:param endTime   => datetime("2024-05-12T14:00:00+10:00");
```

```
// ----- QUERY -----
```

```
MATCH (c:Client)-[:MADE]->(p:Purchase)-[:AT_STORE]->(s:Store)
```

```
WHERE p.timestamp >= $startTime
```

```
    AND p.timestamp < $endTime
```

```
    AND p.amount > 0
```

```
WITH
```

```
    c.name AS name,
```

```
    s.name AS store,
```

```
    SUM(p.amount) AS total_spent,
```

```
    COUNT(p) AS txn_count,
```

```
    MIN(p.timestamp) AS first_txn_time,
```

```
    MAX(p.timestamp) AS last_txn_time
```

```
WITH
```

```
    name,
```

```
    COLLECT({
```

```
        store: store,
```

```
        total_spent: total_spent
```

```
    }) AS store_spends,
```

```

SUM(total_spent) AS total_in_window,
SUM(txn_count) AS txn_count,
MIN(first_txn_time) AS first_txn_time,
MAX(last_txn_time) AS last_txn_time
// Identify each client's top store
WITH name, total_in_window, txn_count, first_txn_time, last_txn_time,
      REDUCE(topStore = {store: null, spend: 0}, ss IN store_spends |
      CASE WHEN ss.total_spent > topStore.spend
            THEN {store: ss.store, spend: ss.total_spent}
            ELSE topStore END
      ) AS top_store_data
WITH
name,
total_in_window,
txn_count,
first_txn_time,
last_txn_time,
top_store_data.store AS top_store,
top_store_data.spend AS top_store_spend
ORDER BY total_in_window DESC

// ----- Manual Dense Rank -----
WITH COLLECT({
name: name,
total_in_window: total_in_window,
txn_count: txn_count,
first_txn_time: first_txn_time,
last_txn_time: last_txn_time,
top_store: top_store,

```

```

top_store_spend: top_store_spend
}) AS clientData

WITH clientData,
  [x IN clientData | x.total_in_window] AS totals,
  apoc.coll.toSet([x IN clientData | x.total_in_window]) AS uniqueTotals
UNWIND clientData AS row
WITH row,
  apoc.coll.indexOf(uniqueTotals, row.total_in_window) + 1 AS rank
WHERE rank = 1 // <-- change to <= 5 if you want the top 5
RETURN
  row.name AS name,
  row.total_in_window AS total_in_window,
  row.txn_count AS txn_count,
  row.first_txn_time AS first_txn_time,
  row.last_txn_time AS last_txn_time,
  row.top_store AS top_store,
  row.top_store_spend AS top_store_spend,
  rank
ORDER BY rank ASC, name;

```

Problem 2:

Top Clients with Largest Negative Cash Flow

// ----- PROBLEM 2: Negative Cashflow Clients -----

// Step 1: calculate all outgoing money (Purchases + Transfers sent)

MATCH (c:Client)

OPTIONAL MATCH (c)-[:MADE]->(p:Purchase)

```
WITH c, COALESCE(SUM(p.amount), 0) AS total_purchases
```

```
OPTIONAL MATCH (c)-[:TRANSFERRED_TO]->(receiver:Client)
```

```
WITH c, total_purchases, COALESCE(SUM(receiver.amount), 0) AS total_transfers_sent
```

```
// Step 2: calculate all incoming transfers
```

```
OPTIONAL MATCH (sender:Client)-[r:TRANSFERRED_TO]->(c)
```

```
WITH c,
```

```
    total_purchases,
```

```
    total_transfers_sent,
```

```
    COALESCE(SUM(r.amount), 0) AS total_transfers_received
```

```
// Step 3: compute cashflow metrics
```

```
WITH
```

```
    c.name AS name,
```

```
    (total_purchases + total_transfers_sent) AS total_outgoing,
```

```
    total_transfers_received AS total_incoming,
```

```
    (total_transfers_received - (total_purchases + total_transfers_sent)) AS balance,
```

```
CASE
```

```
    WHEN total_purchases >= total_transfers_sent THEN total_purchases
```

```
    ELSE total_transfers_sent
```

```
END AS max_outgoing
```

```
// Step 4: count all relationships (transaction volume)
```

```
OPTIONAL MATCH (c)-[tx]->()
```

```
WITH
```

```
    name,
```

```
    total_outgoing,
```

```
    total_incoming,
```

```
balance,  
max_outgoing,  
COUNT(tx) AS txn_count  
ORDER BY balance ASC  
LIMIT 20  // <-- safety limit to keep memory low
```

```
// Step 5: rank manually (no OVER)
```

```
WITH COLLECT({  
  name:name,  
  balance:balance,  
  max_outgoing:max_outgoing,  
  total_outgoing:total_outgoing,  
  total_incoming:total_incoming,  
  txn_count:txn_count  
}) AS data
```

```
UNWIND range(0, size(data)-1) AS i  
WITH data[i] AS row, i+1 AS rank  
WHERE rank <= 5  
RETURN  
  row.name AS name,  
  row.balance AS balance,  
  row.max_outgoing AS max_outgoing,  
  row.total_outgoing AS total_outgoing,  
  row.total_incoming AS total_incoming,  
  row.txn_count AS txn_count,  
  rank  
ORDER BY rank ASC, name;
```

neo4j\$ // ----- PROBLEM 2: Negative Cashflow Clients (Community Edition Safe) ----- // Step 1: calculate all outgoing money (Pu

name	balance	max_outgoing	total_outgoing	total_incoming	txn_count	rank
1 "Justin Cross"	-870624.3782978 493	1304418.2147302 832	1304418.2147302 832	433793.83643243 4	189096	1
2 "Mackenzie Whit ahead"	-529795.4247575 849	529795.42475758 49	529795.42475758 49	0	189096	2
3 "Tyler Travis"	-523156.7662823 6345	523156.76628236 345	523156.76628236 345	0	189096	3
4 "Julian Short"	-490651.5431620 1987	490651.54316201 987	490651.54316201 987	0	189096	4
5 "Eva Simmons"	-466235.9872898 96	707665.73177198 84	707665.73177198 84	241429.74448209 24	189096	5

Started streaming 5 records after 244 ms and completed after 3.157 ms.

Problem 3 :

Detect “Pass-Through” Clients

// ----- PROBLEM 3: Pass-Through Funds Detection (Community Edition Safe - No APOC) -----

```
:param sellerName => "Woods";
```

```
:param lookbackHours => 48;
```

1 Find the suspicious seller node

```
MATCH (targetSeller:Store)
```

```
WHERE toLower(targetSeller.name) CONTAINS toLower($sellerName)
```

2 Find all purchases at that seller

```
MATCH (client:Client)-[p:MADE]->(txn:Purchase)-[:AT_STORE]->(targetSeller)
```

```
WHERE txn.amount > 0
```

3 For each purchase, look back for incoming transfers within the window

```
OPTIONAL MATCH (sender:Client)-[xfer:TRANSFERRED_TO]->(client)
```

```
WHERE xfer.amount > 0
```

```
AND xfer.timestamp < txn.timestamp
```

```
AND duration.between(xfer.timestamp, txn.timestamp).hours <= $lookbackHours  
AND sender <> client // avoid self-funding
```

④ Aggregate and calculate top sender manually

```
WITH client.name AS name,  
      COLLECT(DISTINCT sender.name) AS senders,  
      COUNT(DISTINCT sender) AS n_senders,  
      SUM(DISTINCT xfer.amount) AS attributed_xfer,  
      SUM(DISTINCT txn.amount) AS total_purchase,  
      MIN(txn.timestamp) AS first_purchase_time,  
      // manually find the largest transfer amount  
      REDUCE(maxAmt = 0.0, s IN COLLECT(xfer.amount) |  
              CASE WHEN s > maxAmt THEN s ELSE maxAmt END) AS top_sender_amount  
  
// Find the sender corresponding to that max amount  
OPTIONAL MATCH (sender2:Client)-[xfer2:TRANSFERRED_TO]->(client)  
WHERE xfer2.amount = top_sender_amount  
WITH  
      name,  
      senders,  
      n_senders,  
      attributed_xfer,  
      total_purchase,  
      first_purchase_time,  
      sender2.name AS top_sender,  
      top_sender_amount
```

⑤ Compute percentage and filter suspicious clients

```
WITH name,
```

```

total_purchase,
attributed_xfer,
n_senders,
first_purchase_time,
top_sender,
top_sender_amount,
CASE
    WHEN attributed_xfer > 0 THEN (total_purchase / attributed_xfer) * 100
    ELSE 0
END AS percentage
WHERE percentage >= 5 AND total_purchase > 0 AND n_senders >= 1

```

6 Return final report

```

RETURN
name,
round(percentage,2) AS percentage,
round(attributed_xfer,2) AS attributed_xfer,
round(total_purchase,2) AS total_purchase,
first_purchase_time,
n_senders,
top_sender,
round(top_sender_amount,2) AS top_sender_amount
ORDER BY percentage DESC, total_purchase DESC
LIMIT 20;

```

Problem 4 :

Temporal Transaction Ordering

// ----- PROBLEM 4: Temporal Transaction Ordering -----

① Collect all transactions per client (Purchase + Transfer)

MATCH (c:Client)

OPTIONAL MATCH (c)-[:MADE]->(p:Purchase)

OPTIONAL MATCH (c)-[:TRANSFERRED_TO]->(t1:Transaction)

OPTIONAL MATCH (c)-[:TRANSFERRED_TO]-(t2:Transaction)

WITH c, collect(p) + collect(t1) + collect(t2) AS txns

WHERE size(txns) > 1

② Order transactions by timestamp

UNWIND txns AS t

WITH c, t

WHERE t.timestamp IS NOT NULL

ORDER BY c.name, t.timestamp ASC

WITH c, collect(t) AS ordered_txns

WHERE size(ordered_txns) > 1

③ Create consecutive pairs (txn[i] → txn[i+1])

UNWIND range(0, size(ordered_txns)-2) AS i

WITH c, ordered_txns[i] AS txn1, ordered_txns[i+1] AS txn2

④ Create NEXT_TXN_OF relationships

MERGE (txn1)-[:NEXT_TXN_OF {client: c.name}]->(txn2);

PART 3

Problem 1 :

Community Detection (Louvain + Connected Components)

```
// 🐙 Create projection for community detection
```

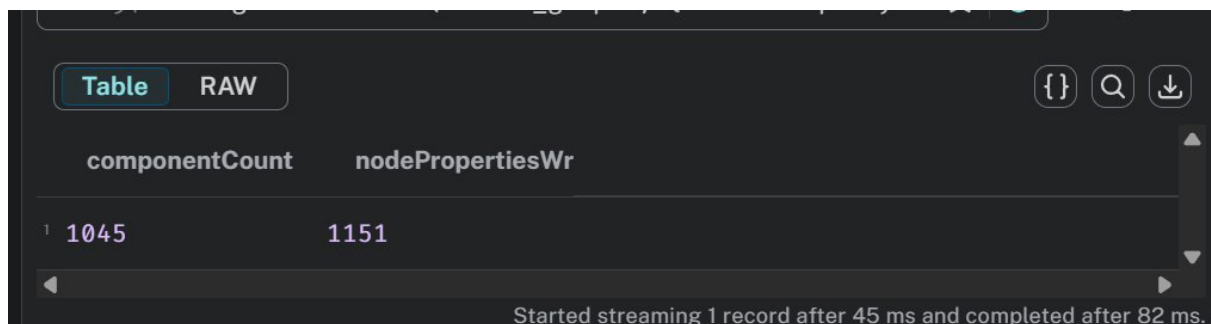
```
CALL gds.graph.project(  
  'fraud_graph',  
  'Client',  
  {  
    SIMILAR_TO: {  
      type: 'SIMILAR_TO',  
      orientation: 'UNDIRECTED'  
    }  
  }  
);
```

Louvain:

```
CALL gds.louvain.write('fraud_graph', {  
  writeProperty: 'louvain_group'  
})  
  
YIELD communityCount, modularity, modularities;
```

Connected Components (WCC):

```
CALL gds.wcc.write('fraud_graph', {  
  writeProperty: 'component_group'  
})  
  
YIELD componentCount, nodePropertiesWritten;
```



	componentCount	nodePropertiesWr
1	1045	1151

Started streaming 1 record after 45 ms and completed after 82 ms.

Identify Large Fraud-Like Clusters (≥ 5 members)

```
// □ Count and label dense clusters

MATCH (c:Client)

WITH c.louvain_group AS group_id, count(*) AS members

WHERE members >= 5

WITH collect(group_id) AS fraud_groups

MATCH (c:Client)

WHERE c.louvain_group IN fraud_groups

SET c:FraudGroup,

    c.fraud_group_id = c.louvain_group,

    c.detection_method = 'Louvain';
```

Verify Group Sizes

```
MATCH (c:FraudGroup)

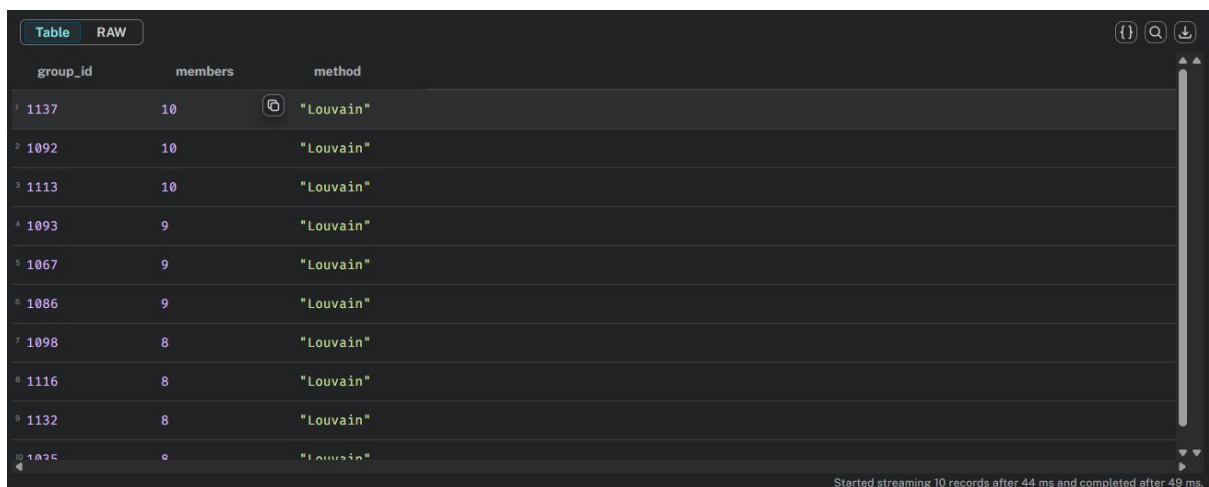
RETURN c.fraud_group_id AS group_id,

    count(*) AS members,

    c.detection_method AS method

ORDER BY members DESC

LIMIT 10;
```



The screenshot shows a database interface with a table view. The table has three columns: group_id, members, and method. It displays 10 rows of data, sorted by the number of members in descending order. The first row has 10 members, and the last row has 6 members. All rows use the 'Louvain' detection method. The interface includes a 'Table' tab, a 'RAW' tab, and a search icon. A status bar at the bottom indicates that 10 records were streamed after 44 ms and completed after 49 ms.

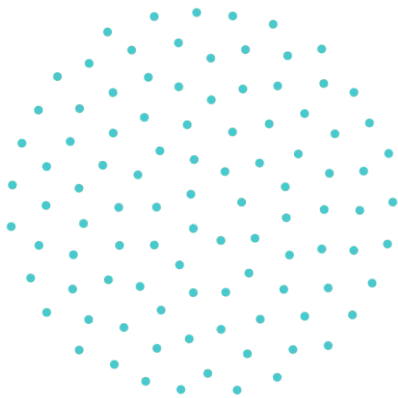
group_id	members	method
1137	10	"Louvain"
1092	10	"Louvain"
1113	10	"Louvain"
1093	9	"Louvain"
1067	9	"Louvain"
1086	9	"Louvain"
1098	8	"Louvain"
1116	8	"Louvain"
1132	8	"Louvain"
1035	6	"Louvain"

Visualise in Neo4j Browser

```
MATCH (c:FraudGroup)
```

```
RETURN c
```

```
LIMIT 100;
```



Problem 2 :

Identify Cross-Boundary Transactions

```
MATCH (src:Client)-[t:TRANSFERRED_TO]->(dst:Client)
```

```
WHERE src.fraud_group_id IS NOT NULL
```

```
AND (dst.fraud_group_id IS NULL OR dst.fraud_group_id <> src.fraud_group_id)
```

```
WITH dst.name AS recipient,
```

```
collect(DISTINCT src.name) AS senders,
```

```
count(DISTINCT src) AS n_senders,
```

```

    sum(t.amount) AS total_incoming
WHERE total_incoming > 0

RETURN recipient, n_senders, round(total_incoming,2) AS total_incoming

ORDER BY total_incoming DESC

LIMIT 10;

```

recipient	n_senders	total_incoming
"Jonathan Palme r"	9	1011361.9
"Addison Muelle r"	10	800715.62
"Juan Williams"	10	751419.26
"Andrew Adkins"	8	714480.63
"Tristan Sosa"	9	234748.2
"Grace Dickerso n"	8	199990.9
"Thomas Spence"	10	193353.77
"Lillian Elliot t"	7	182529.5
"Wyatt Cohen"	8	176777.91
"Jose Wright"	8	132990.7

Boundary Subgraph Projection & Clustering

// Safe version — skips relationships with missing endpoints

```

CALL gds.graph.project.cypher(
  'boundary_graph',
  '
MATCH (c:Client)
WHERE c.fraud_group_id IS NOT NULL
OR EXISTS {
  MATCH (f:Client)-[:TRANSFERRED_TO]->(c)
  WHERE f.fraud_group_id IS NOT NULL
}
RETURN id(c) AS id
',
  '
MATCH (a:Client)-[r:TRANSFERRED_TO]->(b:Client)
WHERE a.fraud_group_id IS NOT NULL

```

```

    OR b.fraud_group_id IS NOT NULL

    RETURN id(a) AS source, id(b) AS target, r.amount AS amount
    ;

    { validateRelationships: false } // ✓ Fix: ignore broken links
);

```

nodeQuery	relationshipQuery	graphName	nodeCount	relationshipCount	projectMillis
<pre> MATCH (c:Client) WHERE c.fraud_group_id IS NOT NULL OR EXISTS { MATCH (f:Client) WHERE f.fraud_group_id IS NOT NULL } RETURN id(c) AS id" </pre>	<pre> MATCH (a:Client) WHERE a.fraud_group_id IS NOT NULL OR b.fraud_group_id IS NOT NULL RETURN id(a) AS source, id(b) AS target, r.amount AS amount" </pre>	boundary_graph	131	117	34

Started streaming 1 record after 11 ms and completed after 56 ms.

Centrality Analysis on Largest Subset

```

// Find the largest boundary cluster

MATCH (c:Client)

WHERE c.boundary_cluster IS NOT NULL

WITH c.boundary_cluster AS cluster_id, count(*) AS size

ORDER BY size DESC LIMIT 1

// Project subgraph for that cluster (modern syntax)

WITH cluster_id

CALL {

  WITH cluster_id

  RETURN gds.graph.project(

    'largest_boundary_cluster',

    ['Client'],

    {

```

```

TRANSFERRED_TO: {
  type: 'TRANSFERRED_TO',
  properties: 'amount'
}
},
{
  nodeProperties: ['boundary_cluster'],
  nodeFilter: 'n.boundary_cluster = $cluster_id',
  relationshipFilter: 'r.amount > 0',
  parameters: { cluster_id: cluster_id },
  validateRelationships: false
}
) AS result
}
YIELD graphName, nodeCount, relationshipCount, projectMillis
RETURN graphName, nodeCount, relationshipCount, projectMillis;

```

PageRank

```

CALL gds.pageRank.write('boundary_graph', {
  relationshipWeightProperty: 'amount',
  writeProperty: 'pagerank_score'
})
YIELD nodePropertiesWritten, configuration, writeMillis, ranIterations, didConverge;

```

	nodePropertiesWr	configuration	writeMillis	ranIterations	didConverge
131		{ writeProperty: "pagerank_score", jobId: "9b604a8f-b7a8-4ad2-89c0-34b67700e7ea", scaler: "NONE", sourceNodes: [], sudo: false, maxIterations: 20, dampingFactor: 0.85, writeToResultStore: false, writeConcurrency: 4, relationshipWeightProperty: "amount", logProgress: true, nodeLabels: ["*"], concurrency: 4, relationshipTypes: ["*"], tolerance: 1e-7 }	3	2	true

Betweenness Centrality:

```
CALL gds.betweenness.write('boundary_graph', {
  writeProperty: 'betweenness_score'
})
```

YIELD nodePropertiesWritten, computeMillis;

Compare the results:

```
// Compare PageRank vs Betweenness
```

```
MATCH (c:Client)
```

```
WHERE c.pagerank_score IS NOT NULL
```

```
AND c.betweenness_score IS NOT NULL
```

```
RETURN
```

```
  c.name AS client,
```

```
  round(c.pagerank_score, 4) AS pagerank_score,
```

```
  round(c.betweenness_score, 4) AS betweenness_score
```

```
ORDER BY pagerank_score DESC
```

```
LIMIT 10;
```

	client	pagerank_score	betweenness_sco
1	"Addison Mueller"	1.425	0.0
2	"Juan Williams"	1.425	0.0
3	"Thomas Spence"	1.425	0.0
4	"Hannah Byers"	1.2975	0.0
5	"Tristan Sosa"	1.2975	0.0
6	"Jonathan Palmer"	1.2975	0.0
7	"Andrew Adkins"	1.17	0.0
8	"Wyatt Cohen"	1.17	0.0
9	"Ethan Aguilar"	1.17	0.0
10	"Grace Dickerson"	1.17	0.0

To identify the most influential accounts within the boundary-subgraph, two centrality measures PageRank and Betweenness Centrality were applied using Neo4j GDS 2.22.

The PageRank algorithm measures how frequently a node is reached through weighted incoming links, giving higher scores to accounts that repeatedly receive transactions from other high-value senders. In the fraud-network context, a high PageRank therefore indicates a **receiver hub**.

In contrast, Betweenness Centrality quantifies how often a node lies on the shortest transaction paths between other clients. Accounts with high betweenness typically serve as **brokers or routers**, bridging otherwise disconnected clusters and enabling money to circulate while avoiding direct sender-receiver exposure.

Some Screen Shots of my Neo4j

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

neo4j Desktop Feedback Fully-managed cloud database Neo4j AuraDB

Instance: FraudDetectionDB Database: neo4j_cypher_5 User: neo4j

Data services

- Local instances
- Remote connections
- Import

Tools

- Query
- Explore

About

- Settings

Database information

Nodes (37,049)

Relationships (223,901)

Property keys

amount betweenness_score business_hours client client_id component_group data detection_method email flagged fraud_group_id hour id louvain_group name nodes pagerank_score phone relationships style tfn timeOffset timestamp txn_id visualisation weekday

Show less

Last update: 1:25:59 PM

neo4j\$

neo4j\$ // Compare

Table RAW

client	pagerank_score
"Juan Williams"	1.425
"Thomas Spence"	1.425
"Hannah Byers"	1.2975
"Tristan Sosa"	1.2975
"Jonathan Palmer"	1.2975
"Andrew Adkins"	1.17
"Wyatt Cohen"	1.17
"Ethan Aguilar"	1.17
"Grace Dickerson"	1.17

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

neo4j Desktop Feedback Fully-managed cloud database Neo4j AuraDB

Instance: FraudDetectionDB Database: neo4j_cypher_5 User: neo4j

Data services

- Local instances
- Remote connections
- Import

Tools

- Query
- Explore

About

- Settings

Database information

Nodes (37,049)

Relationships (223,901)

Property keys

amount betweenness_score business_hours client client_id component_group data detection_method email flagged fraud_group_id hour id louvain_group name nodes pagerank_score phone relationships style tfn timeOffset timestamp txn_id visualisation weekday

Show less

neo4j\$

Started streaming 1 record after 6 ms and completed after 13 ms.

neo4j\$ LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row RETURN row

LOAD CSV WITH HEADERS FROM 'file:///clients.csv' AS row RETURN row

LOAD CSV WITH HEADERS FROM 'file:///stores.csv' AS row RETURN row

LOAD CSV WITH HEADERS FROM 'file:///purchase.csv' AS row RETURN row

LOAD CSV WITH HEADERS FROM 'file:///xfer.csv' AS row RETURN row

neo4j\$ LOAD CSV WITH HEADERS FROM 'file:///purchase.csv' AS row RETURN row

No changes, no records

Completed after 3,897 ms

03N90: Cartesian product

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

Instance: FraudDetectionDB Database: neo4j CYPHER 5 User: neo4j

Data services

- Local instances
- Remote connections
- Import

Tools

- Query
- Explore

About

- Settings

Database information

Nodes (37,049)

- Client FraudGroup
- Purchase Source Store
- Transaction

Relationships (223,901)

- AT_STORE
- FROM_SOURCE MADE
- NEXT_TXN_OF PURCHASED
- SIMILAR_TO
- TRANSFERRED_TO

Property keys

- amount betweenness_score
- business_hours client
- client_id component_group
- data detection_method email
- flagged fraud_group_id hour

neo4j\$

```
neo4j$ MATCH (a)-[r]->(b) RETURN a.name AS From, type(r) AS
```

Table RAW

	From	Relationship	To	Amount
1	"Sebastian Stei n"	"PURCHASED"	"QuickLink Comp any"	13860.134554298444
2	"Sebastian Stei n"	"PURCHASED"	"QuickLink Comp any"	4906.779450088262
3	"Sebastian Stei n"	"PURCHASED"	"QuickLink Comp any"	2787.370510339309
4	"Sebastian Stei n"	"PURCHASED"	"QuickLink Comp any"	10509.720426087357
5	"Sebastian Stei n"	"PURCHASED"	"QuickLink Comp any"	2455.3224552316124
6	"Sebastian Stei n"	"PURCHASED"	"QuickLink Comp any"	23430.29755411354
7	"Sebastian Stei n"	"PURCHASED"	"Felics LLC"	8842.7565242114

Last update: 1:28:30 PM

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

Instance: FraudDetectionDB Database: neo4j CYPHER 5 User: neo4j

Data services

- Local instances
- Remote connections
- Import

Tools

- Query
- Explore

About

- Settings

Database information

Nodes (37,049)

- Client FraudGroup
- Purchase Source Store
- Transaction

Relationships (223,901)

- AT_STORE
- FROM_SOURCE MADE
- NEXT_TXN_OF PURCHASED
- SIMILAR_TO
- TRANSFERRED_TO

Property keys

- amount betweenness_score
- business_hours client
- client_id component_group
- data detection_method email
- flagged fraud_group_id hour

neo4j\$

```
neo4j$ MATCH (t:Transaction:Purchase) RETURN t LIMIT 5;
```

Graph Table RAW

Node details

Purchase Transaction

Key	Value
<id>	4:543272e5-bf7a-45bf-9c52-57dd16f83fc0:1303
amount	594.3516815829278
business_hours	false
hour	0
timestamp	1970-01-01T00:00:00.268000000Z
txn_id	"1c40fa9d-72d6-41e6-a8f7-3b09420061d5"
weekday	"Thu"

Last update: 1:30:00 PM

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

Instance: FraudDetectionDB Database: neo4j CYPHER 5 User: neo4j

Data services

- Local instances
- Remote connections
- Import

Tools

- Query
- Explore

About

- Settings

Database Information

Nodes (37,049)

- Client FraudGroup
- Purchase Source Store
- Transaction

Relationships (223,901)

- AT_STORE
- FROM_SOURCE MADE
- NEXT_TXN_OF PURCHASED
- SIMILAR_TO
- TRANSFERRED_TO

Property keys

- amount betweenness_score
- business_hours client
- client_id component_group
- data detection_method email
- flagged fraud_group_id hour
- id timestamp timestamp

Last update: 1:30:30 PM

neo4j\$

- CREATE CONSTRAINT unique_txn_id IF NOT EXISTS
- // Add indexes for frequently filtered or joined fields
- CREATE INDEX client_tfn_index IF NOT EXISTS FOR (c:Client) ON (c.tfn)
- CREATE INDEX txn_timestamp_index IF NOT EXISTS FOR (t:Transaction) ON (t.timestamp)
- CREATE INDEX store_name_index IF NOT EXISTS FOR (s:Store) ON (s.name)

neo4j\$ // Create Source nodes MERGE (s1:Source {name:'purchase'})

- // Create Source nodes
- // Link Purchase transactions to their source
- // Link Transfer transactions to their source

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

Feedback Fully-managed cloud database Neo4j AuraDB

Instance: FraudDetectionDB Database: neo4j CYPHER 5 User: neo4j

Data services

- Local instances
- Remote connections
- Import

Tools

- Query
- Explore

About

- Settings

Database information

Nodes (37,049)

- Client FraudGroup
- Purchase Source Store
- Transaction

Relationships (223,901)

- AT_STORE
- FROM_SOURCE MADE
- NEXT_TXN_OF PURCHASED
- SIMILAR_TO
- TRANSFERRED_TO

Property keys

- amount betweenness_score
- business_hours client
- client_id component_group
- data detection_method email
- flagged fraud_group_id hour
- id timestamp timestamp

Last update: 1:32:00 PM

neo4j\$

neo4j\$ RETURN gds.version();

Table RAW

gds.version()

"2.22.0"

Started streaming 1 record after 34 ms and completed after 35 ms.

neo4j\$ // ----- PROBLEM 4: Temporal Transaction Ordering

- Created 34,805 relationships, set 34,805 properties Completed after 1,081 ms

neo4j\$ // ----- PROBLEM 4: Temporal Transaction Ordering

- 42152: Syntax error or access rule violation - no longer valid syntax**
The property existence syntax `... exists(variable.property)` is no longer supported. Please use `variable.property IS NOT NULL` instead.
[Show less](#)
- 42001: Syntax error or access rule violation - invalid syntax**

neo4j\$ // ----- PROBLEM 4: Temporal Transaction Ordering

Neo4j Desktop 2.0.3
File Edit View Window Help Developer

neo4j Desktop

Feedback Fully-managed cloud database Neo4j AuraDB

Data services

Local instances

Remote connections

Import

Tools

Query

Explore

About

Settings

Instance: FraudDetectionDB Database: neo4j_cypher_5 User: neo4j

Database information

Nodes (37,049)

Client

FraudGroup

Purchase

Source

Store

Transaction

Relationships (223,901)

AT_STORE

FROM_SOURCE

MADE

NEXT_TXN_OF

PURCHASED

SIMILAR_TO

TRANSFERRED_TO

Property keys

amount

betweenness_score

business_hours

client

client_id

component_group

data

detection_method

email

flagged

fraud_group_id

hour

latitude

longitude

name

password

score

source

store

time

transaction_id

transaction_type

type

value

weight

year

neo4j\$

neo4j\$ MATCH (c:Client) RETURN c LIMIT 100;

Graph Table RAW

Results overview

Nodes (100)

Client (100)

FraudGroup (100)

Last update: 1:33:31 PM

Started streaming 100 records after 29 ms and completed after 33 ms.